Lab: Configuring HTTPS with Nginx and TLS/SSL

Introduction:

The Goals Of The Lab:

The objective of this lab is to deploy a local Nginx web server using Docker and Docker Compose to serve a static portfolio website. Network traffic is analyzed using Wireshark to observe data traveling in plaintext over HTTP. Finally, OpenSSL is used to generate a self-signed SSL/TLS certificate to transition the server to secure HTTPS, verifying the structural differences within the TLS handshake records.

Purpose of HTTPS/TLS:

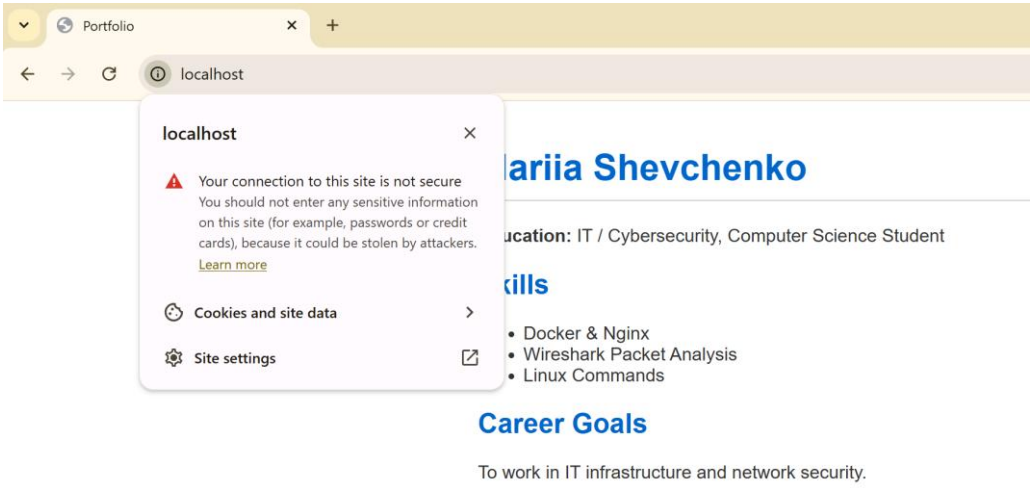
HTTPS is the secure version of HTTP. It uses the TLS protocol to protect the data moving between the browser and a website.

HTTPS/TLS protects connections using three simple rules:

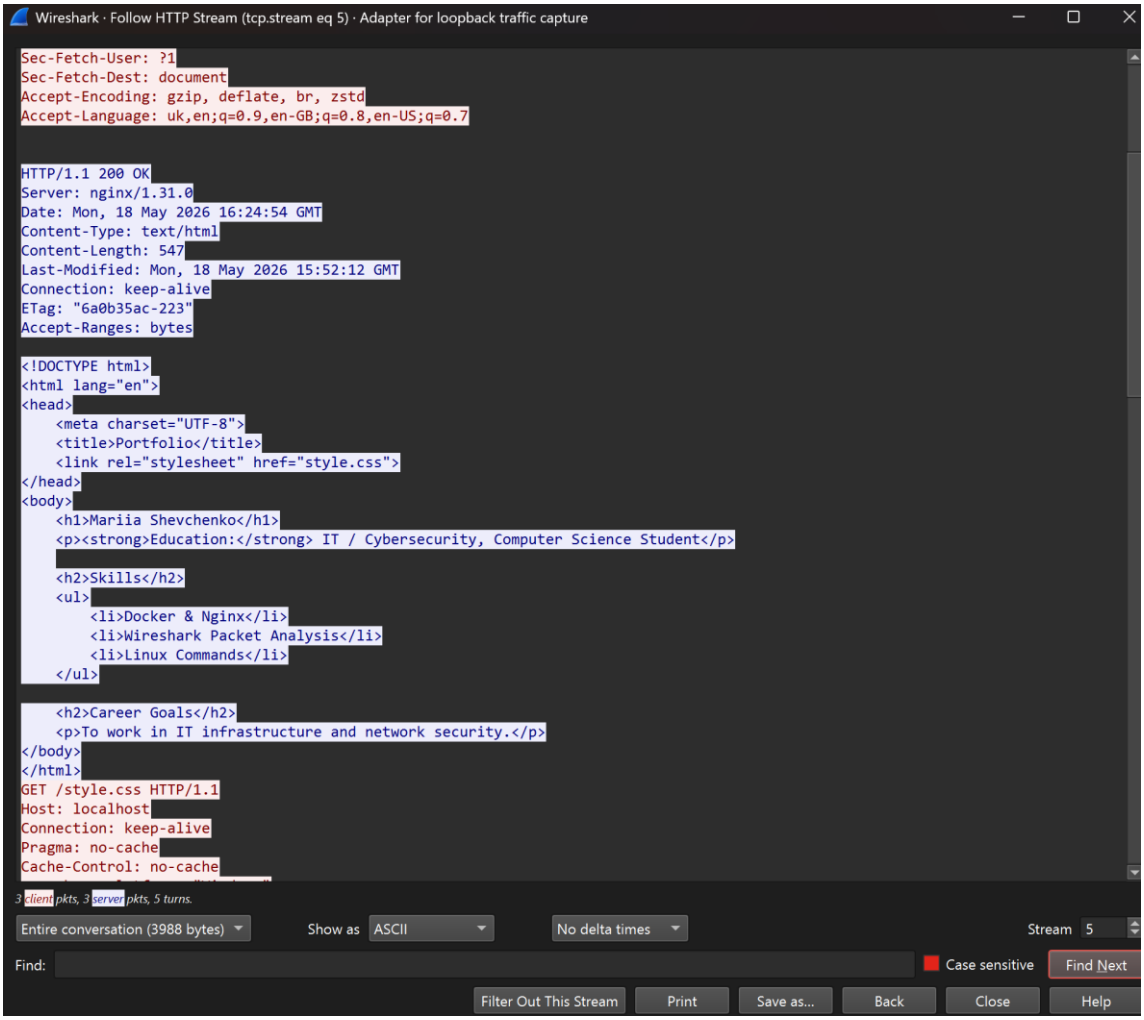
- **Confidentiality:** Standard HTTP sends data in plain text, meaning anyone on your network can read it. TLS encrypts this data so that even if someone steals a packet, they cannot read it.
- **Integrity:** TLS ensures no one alters your data or injects a virus while it travels over the internet. If someone tries to change the data, the browser spots it instantly and cuts the connection.
- **Authentication:** HTTPS uses digital certificates to prove a website is real.

Screenshots:

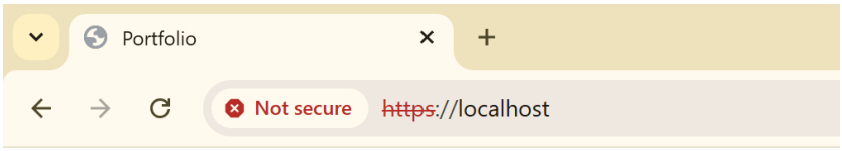
Portfolio website over HTTP:



Unencrypted content:



Portfolio website over HTTPS:



Mariia Shevchenko

Education: IT / Cybersecurity, Computer Science Student

Skills

- Docker & Nginx
- Wireshark Packet Analysis
- Linux Commands

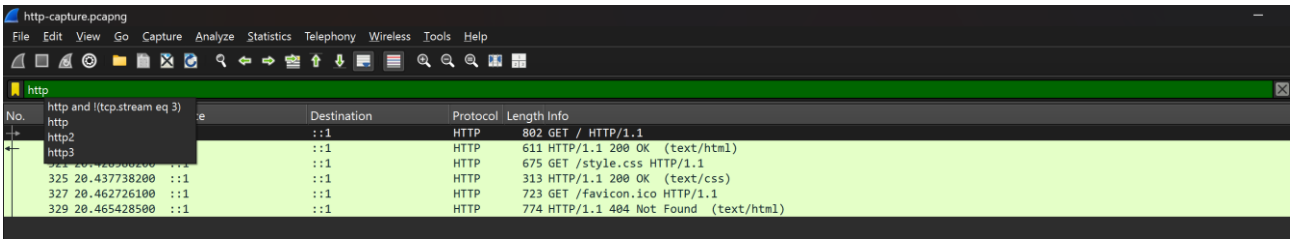
Career Goals

To work in IT infrastructure and network security.

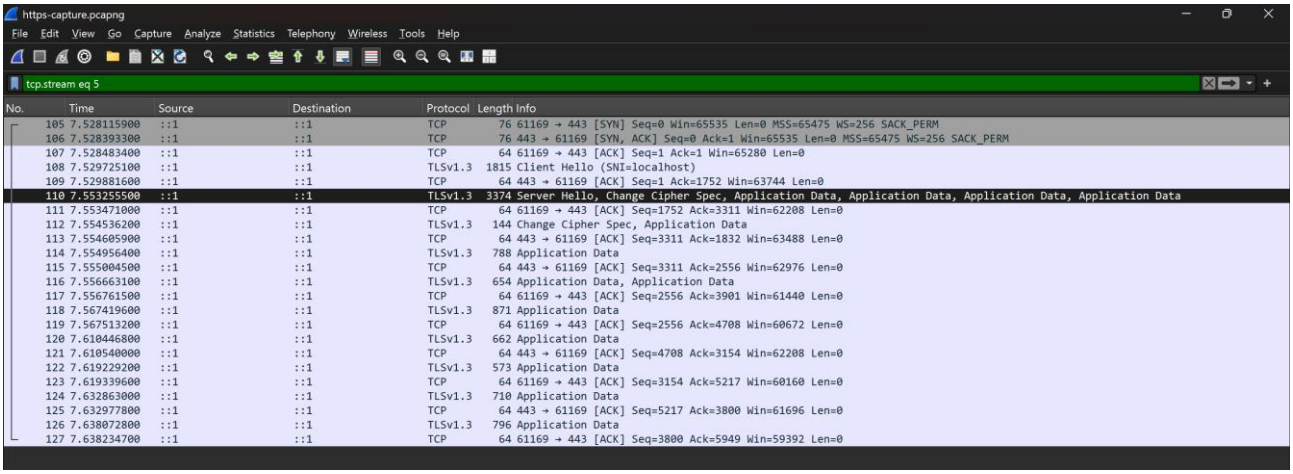
Docker Container Running:

```
PS C:\Users\shevc\portfolio-lab> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS          PORTS
                     NAMES
77987e05e772   nginx:alpine   "/docker-entrypoint.…"   45 minutes ago
Up 45 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tc
p, [::]:443->443/tcp   nginx-web-server
3c29fb4748fe   nginx          "/docker-entrypoint.…"   6 days ago
Up 6 days       0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
                     mynginx
PS C:\Users\shevc\portfolio-lab>
```

Wireshark HTTP capture:



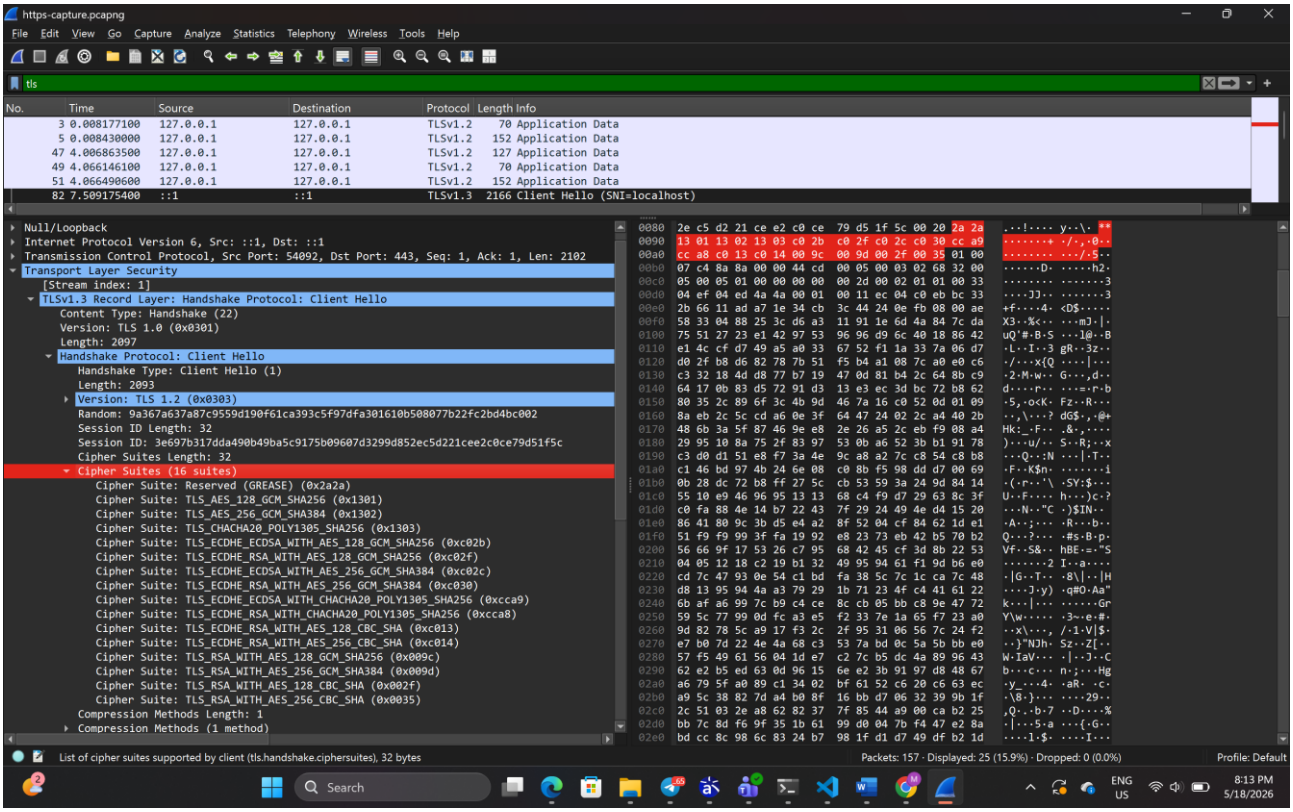
Wireshark HTTPS capture:



TLS Handshake Analysis:

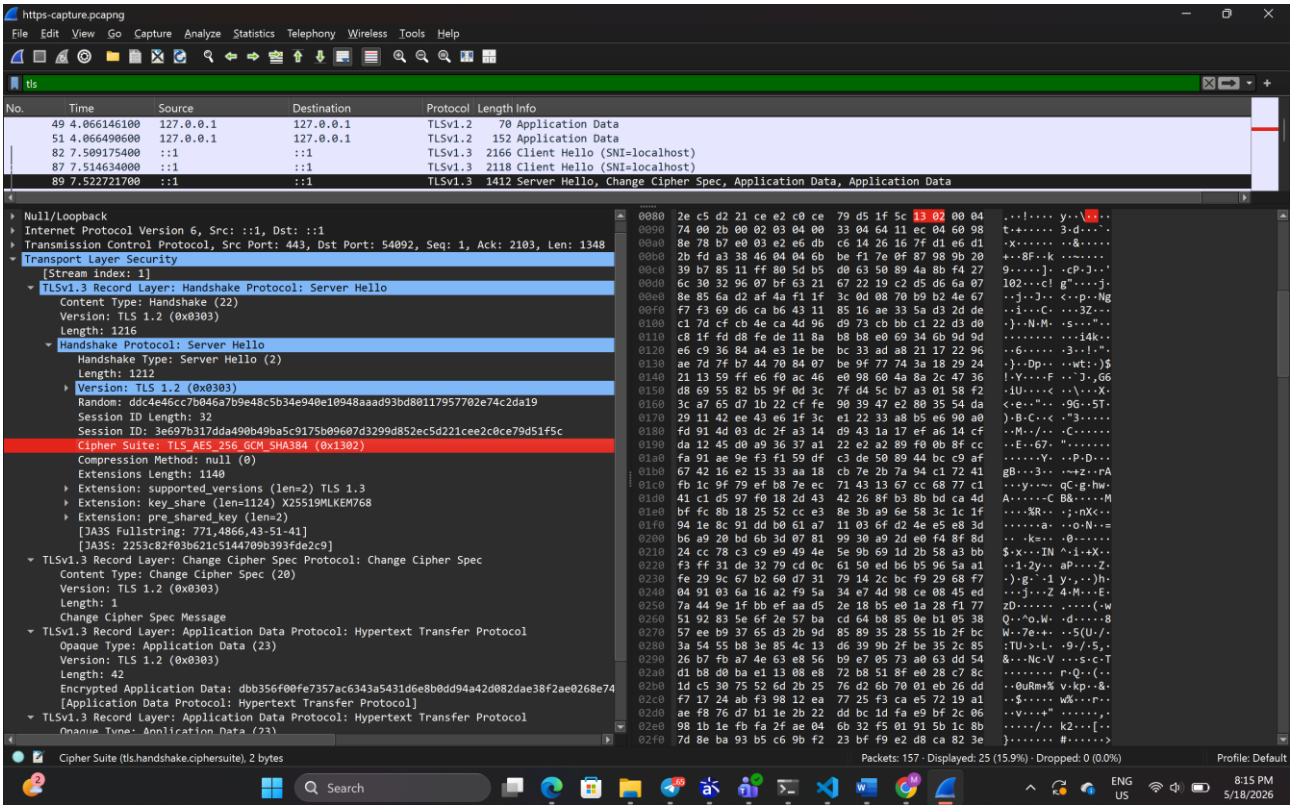
1. Client Hello:

The browser (Client) initiates the secure connection by sending a "Client Hello" packet to the Nginx server. It tells the server what cryptographic capabilities it has, including supported TLS versions, a list of compatible Cipher Suites (encryption algorithms), and a randomly generated number used later to create encryption keys.



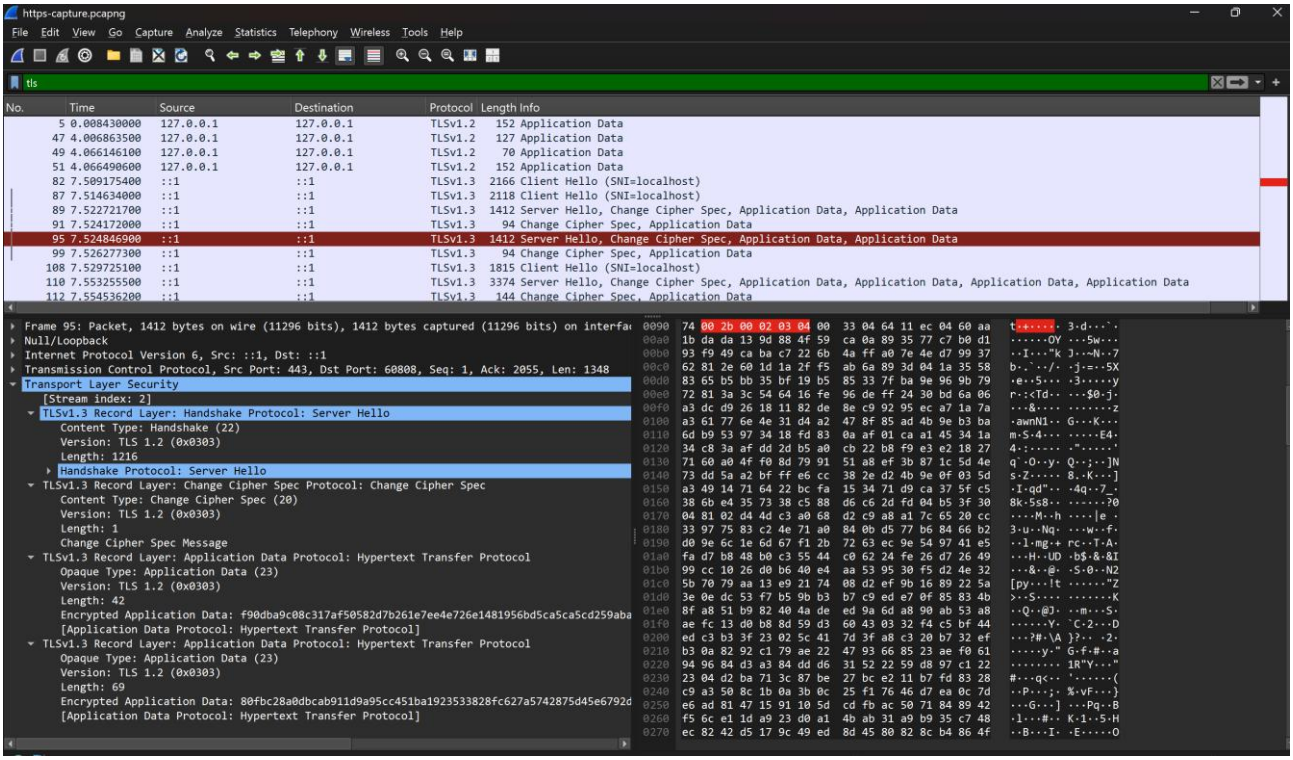
2.Server Hello:

The Nginx server responds with a "Server Hello" packet. After looking at the client's options, the server makes the final decision for the secure tunnel. It selects the highest mutually supported TLS version and chooses the specific Cipher Suite that will be used to encrypt the session.



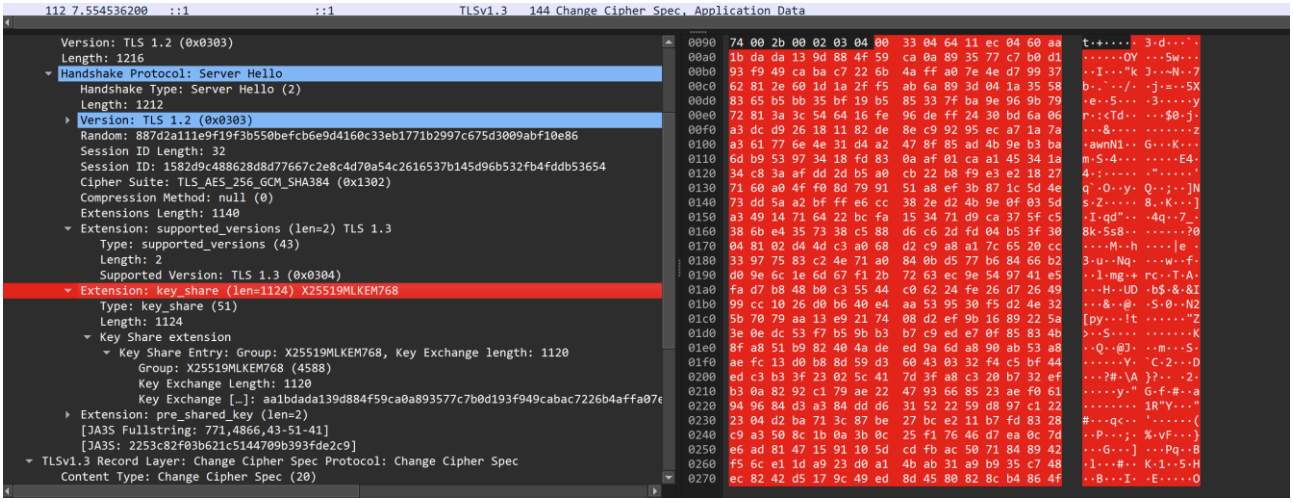
3.Certificate Exchange:

The server negotiated **TLS 1.3**. Unlike TLS 1.2, which transmits the server certificate in cleartext, TLS 1.3 encrypts the Certificate Exchange stage to protect user privacy. The certificate is transmitted securely inside the encrypted handshake containers labeled by Wireshark as **Encrypted Application Data** immediately following the Server Hello.



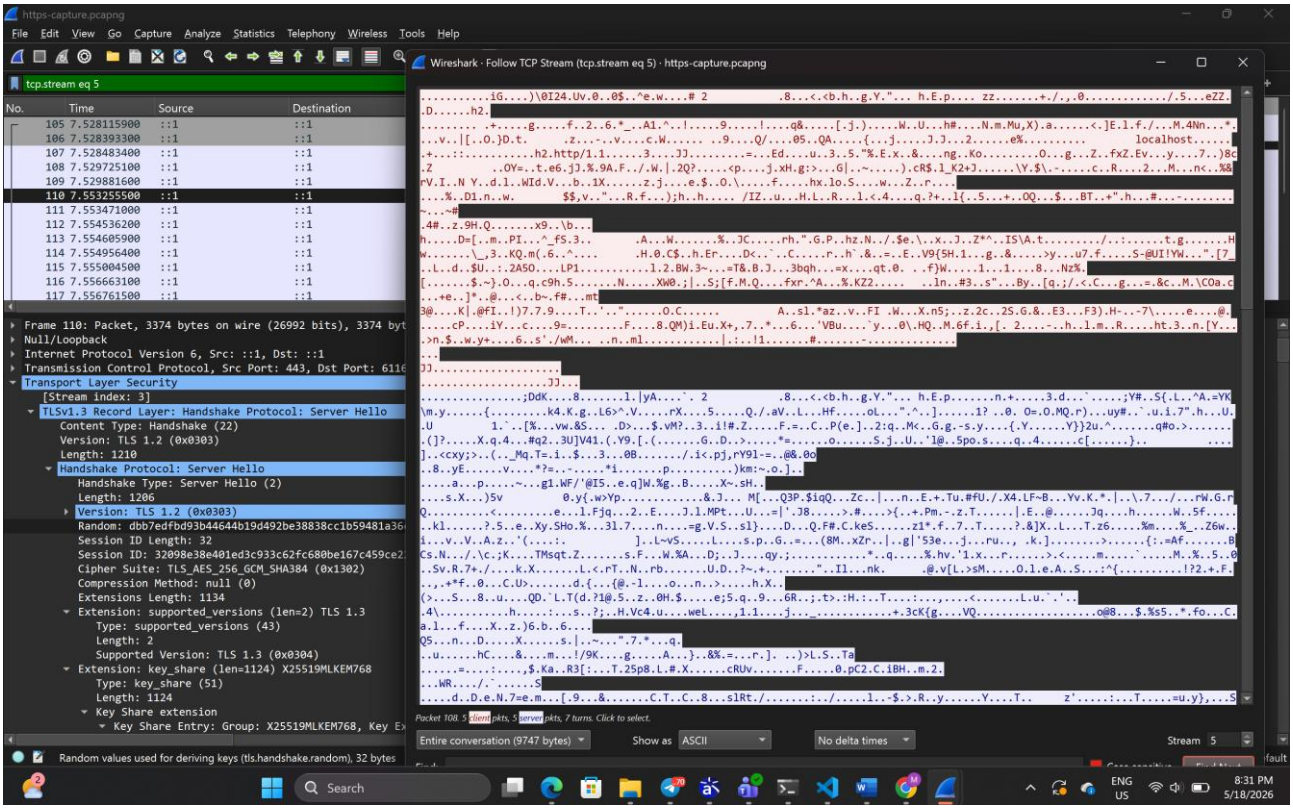
4.Key Exchange:

Now, both sides must agree on a shared secret key without actually sending the key over the air. This could be achieved by using Diffie-Hellman Key Exchange. The server sends its public key parameters, the client sends its public key parameters, and they both calculate the exact same secret key locally.



5.Encrypted Application Data:

From this point forward, every piece of data moving between the browser and Nginx is packed inside an unreadable cryptographic wrapper. Eavesdroppers can only see that data is moving, not what it says.



Reflection:

1.Difference between HTTP and HTTPS: The core difference is security. HTTP transmits data in clear text, meaning anyone monitoring the network can see everything. HTTPS adds a security layer (TLS/SSL) that completely scrambles the data before sending it. Additionally, while HTTP uses port 80 by default, HTTPS uses port 443 to establish its secure, encrypted connections.

2.Benefits of Encryption: Encryption transforms readable data into a secret code that can only be unlocked with the correct key, providing vital security for modern web traffic. Its main benefit is protecting user privacy by ensuring unauthorized people cannot read sensitive personal information.

3.Risks of Unencrypted Traffic: Running an unencrypted HTTP site exposes users to severe security vulnerabilities because data is transmitted in plain text. This makes the connection highly vulnerable to eavesdropping and packet sniffing, allowing attackers to easily read personal information or passwords using tools like Wireshark.

Conclusion:

1.What you learned: This lab practically demonstrated how easy it is to intercept cleartext data on an insecure network, and how to fix it by configuring a containerized web environment. I learned how to deploy an Nginx server using Docker Compose and map secure ports to transition a website from HTTP to HTTPS.

2.Importance of TLS/SSL: It provides a standard, reliable framework that ensures data cannot be modified or read by outside actors while moving from a user's computer to a remote server.

3. How HTTPS improves confidentiality and trust: HTTPS guarantees **confidentiality** by turning readable web data into unreadable ciphertext, ensuring that only the intended recipient can decode it. It builds **trust** by providing digital proof of a website's identity. When users see a valid HTTPS connection, they know their privacy is protected and that they are connected to a legitimate server, creating a safe and reliable browsing experience.